RiskKV-Block: Risk-Constrained Memory Granularity Routing for Long-Context Inference

Yiming Luo
Fudan University
yimingluo@fudan.edu.cn

Abstract

Long-context language models incur substantial online decoding cost because each generated token attends to a large key-value (KV) cache. Existing cache compression methods typically use a fixed cache budget, a fixed granularity, or layer-wise pruning rules. However, we find that the optimal memory granularity is strongly task- and length-dependent: small blocks reduce cost for precise evidence retrieval, medium blocks preserve multi-evidence locality, and larger blocks are necessary for hard long-context cases. We propose RiskKV-Block, a risk-constrained memory controller that dynamically selects a memory action consisting of block size, top-k evidence budget, and a fallback mode. Given a query and a long context, RiskKV-Block scores candidate blocks using evidence-aware lexical and identifier overlap features, routes each example to the smallest safe memory action, and deploys the selected spans either as prompt-level memory or as cache-native KV blocks with RoPE-aware repacking. On a Qwen3-8B evaluation suite covering LongBench-style tasks and RULER 4k/8k/16k retrieval stress tests, a small-block router improves score from 0.8116 to 0.8634 while reducing active tokens to 7.82% and achieving 1.80× end-to-end speedup on a 96-example suite. A larger m10 validation shows that unconstrained small-block routing is unsafe, but a risk-floor action lattice restores full-level quality while using 18.11% active tokens and achieving 1.58× end-to-end speedup. Cache-native experiments show that RoPE-aware repacking is essential: naive KV gathering collapses on 8k/16k RULER, whereas RoPE-aware compact decoding preserves full-cache quality at 6–13% KV. These results suggest that adaptive memory granularity is a key missing dimension in efficient long-context inference.

1 Introduction

Transformer decoding over long contexts is dominated by repeated attention to the KV cache. For a prompt of length n and generated continuation length m, full-cache decoding repeatedly attends to $O(n)$ cached states per generated token. This cost is particularly problematic in long-context serving, where the full-context prefill has already materialized the KV cache and the remaining question is how much of that cache must remain active during online decoding.

Prior work has made strong progress on KV cache efficiency. Heavy-hitter and attention-based methods retain tokens with large attention contribution (Zhang et al., 2023; Li et al., 2024); streaming methods preserve recent tokens and attention sinks for unbounded generation (Xiao et al., 2023); layer-adaptive methods allocate non-uniform cache budgets across Transformer layers (Cai et al., 2024; Yang et al., 2024; Zhou et al., 2024). These methods primarily ask which tokens or layers to keep under a fixed or globally specified cache budget. In contrast, our experiments indicate that memory granularity itself is a major source of quality–efficiency trade-off. On LongBench-style retrieval QA, 128-token blocks can outperform 512-token blocks while using fewer tokens. On RULER 4k/8k, 256-token blocks are the smallest reliable full-quality granularity. On RULER 16k, 64-token blocks

can match full-cache average score at 4.8% active tokens, while 512-token blocks are needed for conservative 100% accuracy.

This motivates a different view: long-context inference should route each query to a memory action that jointly chooses block size, top-k budget, and a fallback policy. We formalize this as risk-constrained memory granularity routing. Given a context x, query q, and candidate action $a = (b, k, f)$ consisting of block size b, top-k budget k, and fallback mode f, the system should minimize active memory subject to a constraint that the compressed execution remains behaviorally close to full-cache execution.

We propose RiskKV-Block, a lightweight controller for this problem. RiskKV-Block first keeps a recent raw window, partitions the older context into candidate block granularities, scores blocks with query-conditioned evidence features, and selects top-k blocks in original order. A router then predicts the smallest safe action from task, length, retrieval-gap, and stability features. For deployment, the same action can be realized in two modes: as prompt-level selected spans for immediate evaluation, or as cache-native KV block selection followed by RoPE-aware repacking and compact decoding. The second mode preserves the serving assumption that the full-context prefill has already occurred and avoids rebuilding a prompt from scratch.

Our current experiments support three claims. First, block-size routing is effective in practice: on a 96-example mixed suite, a trained router improves score over full raw context (0.8535 vs. 0.8116), reduces active tokens to 11.04%, and reaches 1.756× end-to-end speedup. Second, variable granularity is necessary: no single block size dominates across LongBench, RULER 4k, RULER 8k, and RULER 16k. Third, cache-native deployment requires RoPE-aware repacking: at the same KV ratio, naive KV gathering fails on long-context retrieval, while RoPE-corrected repacking recovers full quality.

The paper is currently written as a submission-ready draft with clearly marked experimental placeholders. The remaining work is to fill in large-scale multi-model runs, standard baselines, and full benchmark splits.

2 Related Work

KV cache compression. H_2O retains recent tokens and heavy hitters that dominate attention scores, showing that a small fraction of tokens can preserve generation quality while reducing memory (Zhang et al., 2023). SnapKV observes that important prompt features can be identified from an observation window before generation and compresses the cache by selecting clustered positions (Li et al., 2024). PyramidKV and PyramidInfer exploit layer-wise differences in attention concentration to allocate cache budgets non-uniformly across layers (Cai et al., 2024; Yang et al., 2024). DynamicKV further adapts budgets to task-specific activation patterns (Zhou et al., 2024). RiskKV-Block is complementary: instead of only deciding token retention or layer budgets, it routes among different memory granularities and top-k evidence budgets.

Streaming and long-context inference. StreamingLLM shows that retaining attention sinks and recent tokens enables stable generation beyond the training context window (Xiao et al., 2023). Other long-context systems use sliding windows, recurrent memory, or retrieval-augmented generation to control cost. RiskKV-Block targets query-conditioned long-context inference, where evidence can be selected from a materialized long context and the active cache can be reduced for the online decoding phase.

Long-context evaluation. LongBench provides a multi-task benchmark covering single-document QA, multi-document QA, summarization, few-shot learning, synthetic tasks, and code completion (Bai et al., 2023). RULER is a configurable synthetic benchmark for long-context retrieval, variable tracking, aggregation, and QA stress tests (Hsieh et al., 2024). Our current evaluation focuses on LongBench-style retrieval and summarization tasks plus RULER 4k/8k/16k; the full paper should extend these to full splits and additional model families.

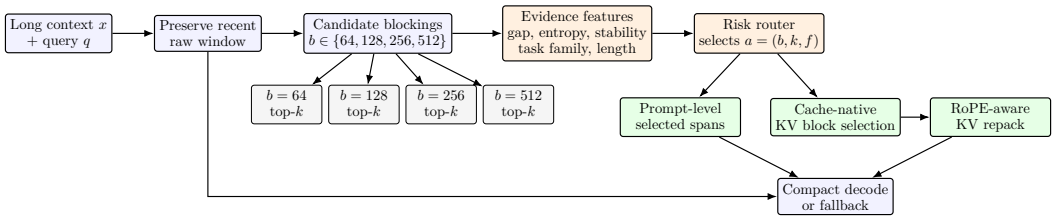

Figure 1: RiskKV-Block routes each query to a memory action that jointly specifies block size, top-k evidence budget, and fallback. The same routed action can be executed as prompt-level selected spans for rapid evaluation or as cache-native KV block selection followed by RoPE-aware repacking for serving.

3 Problem Formulation

Let $x = (x_1, \ldots, x_n)$ be a long context and q be a query. A standard decoder performs full-context prefill and then generates using the complete KV cache:

$$(K, V) = \text{Prefill}_\theta(x, q), \qquad y_{\text{full}} \sim p_\theta(\cdot \mid q, K, V). \tag{1}$$

The goal is to construct an active memory $M_a(x, q)$ under an action $a \in \mathcal{A}$ such that generation remains close to the full-cache output while reducing active memory and latency. We define a candidate action as

$$a = (b, k, f), \tag{2}$$

where b is a block size, k is the number of selected evidence blocks, and f is an optional fallback mode. The active memory ratio is

$$\rho(a; x, q) = \frac{|M_a(x, q)|}{|M_{\text{full}}(x, q)|}. \tag{3}$$

The ideal action minimizes cost subject to a risk constraint:

$$a^\star = \arg\min_{a \in \mathcal{A}} \text{cost}(a; x, q) \quad \text{s.t.} \quad \text{risk}(a; x, q) \le \alpha, \tag{4}$$

where risk measures the probability that the compressed-memory output fails to match the full-memory behavior or task label. In practice, we instantiate this objective through oracle labels from sweep experiments:

$$a_i^\star = \arg\min_{a \in \mathcal{A}} \rho(a; x_i, q_i) \quad \text{s.t.} \quad \text{Score}(a; x_i, q_i) \ge \tau_i. \tag{5}$$

The threshold τ_i can be full-score matching, task-specific full accuracy, or a relaxed cost-strict threshold.

4 Method

4.1 Overview

Figure 1 summarizes RiskKV-Block. The method has four stages: recent-window preservation, evidence block scoring, risk-constrained action routing, and compact execution.

4.2 Evidence Block Scoring

For a block size b, we tokenize the older context and partition it into blocks

$$\mathcal{B}_b(x) = \{B_1^{(b)}, \ldots, B_{N_b}^{(b)}\}. \tag{6}$$

The current implementation uses a lightweight evidence score:

$$s_i(q, b) = |\mathcal{W}(q) \cap \mathcal{W}(B_i^{(b)})| + \lambda |\mathcal{I}(q) \cap \mathcal{I}(B_i^{(b)})|, \tag{7}$$

where $\mathcal{W}(\cdot)$ extracts lower-cased non-stopword English content terms and $\mathcal{I}(\cdot)$ extracts number-bearing identifiers such as 8090293, case123, or A12B. We use $\lambda = 3$ because exact identifiers and numbers are frequently the evidence key in RULER, passage retrieval, and needle-style tasks. Blocks are sorted by s_i, top-k positive-scoring blocks are selected, and selected blocks are restored in original order before execution.

This scorer is deliberately simple. It makes the routing policy transparent and cheap, and it can be replaced by a learned retriever or dense scorer. The paper should include a learned-retriever ablation in the final version.

4.3 Risk-Constrained Router

For each example, we compute features $\phi(x, q)$ including context length, query length, task family, candidate token ratios, top-score gap, score dispersion, top-k stability, number density, and whether high-scoring evidence appears near the recent window. A small MLP router predicts a distribution over actions:

$$p_\psi(a \mid x, q) = \mathrm{softmax}(g_\psi(\phi(x, q))). \tag{8}$$

The router is distilled from sweep-derived oracle labels in Eq. 5. The deployment action is

$$\hat{a} = \arg\max_{a \in \mathcal{A}} p_\psi(a \mid x, q), \tag{9}$$

with a conservative override for task families known to require aggregation or generation rather than sparse evidence selection.

4.4 Fallback Modes

Fallback is a first-class component rather than an exception handler. If evidence selection is uncertain or the task is poorly served by sparse spans, the router can increase top-k, increase block size, use a summary memory, or return to full memory. Examples include:

- Granularity fallback: $b = 64 \to b = 256 \to b = 512$.
- Budget fallback: $k = 4 \to k = 8 \to k = 12$.
- Task fallback: summary/count/scientific QA use summary or full memory rather than sparse retrieval.
- Verifier fallback: output-level verification can trigger a larger action when the small action is unsafe.

4.5 Cache-Native Execution and RoPE-Aware Repacking

Prompt-level selected spans provide a fast evaluation path, but the intended serving mode is cache-native. After full-context prefill, an action selects token spans in the materialized KV cache. Since RoPE encodes absolute token position into queries and keys, naively gathering old keys and then decoding from compact query positions breaks relative position consistency. Let a selected key at original position p be

$$k_p^{\mathrm{rope}} = R(p)k_p, \tag{10}$$

where $R(p)$ is the RoPE rotation. If the key is repacked to compact position $\pi(p)$, RiskKV-Block applies

$$\tilde{k}_{\pi(p)} = R(\pi(p))R(p)^{-1}k_p^{\mathrm{rope}}. \tag{11}$$

Values are copied without RoPE correction. The query position then continues from the compact KV length. This correction is essential: our ablation shows that naive gather collapses on long-context retrieval, while RoPE-aware repacking preserves full-cache quality at the same KV ratio.

5 Experimental Setup

Models. Current experiments use Qwen3-8B with a LoRA adapter trained in the project pipeline. The final submission should add at least one more model family, ideally Llama-3.1-8B-Instruct or Mistral-7B-Instruct.

Table 1: Prompt-level block-size routing results. Free small-block routing is measured on the 96-example sweep suite; risk-floor v2 is measured on the larger m10 validation suite.

Method	Samples	Score	Token ratio	E2E speed
Full raw (m3)	96	0.8116	100.00%	1.000×
Fixed top3 (m3)	96	0.8107	18.22%	1.669×
Block router v1 (m3)	96	0.8535	11.04%	1.756×
Free small-block router (m3)	96	0.8634	7.82%	1.80×
Risk-floor v2 (m10)	320	0.8398	18.11%	1.58×

Table 2: Block-size/top-k frontier. Attention upper is approximated as inverse active token ratio.

Setting	Best action	Score	Token ratio	E2E speed	Attention upper
LongBench	b128 top12	0.3294	15.5%	1.58×	6.4×
LongBench cost-safe	b128 top6	0.2877	10.9%	1.63×	9.2×
RULER 4k	b256 top3	1.0000	30.6%	1.33×	3.3×
RULER 8k	b256 top3	1.0000	15.3%	1.70×	6.5×
RULER 16k conservative	b512 top3	1.0000	10.7%	1.96×	9.3×
RULER 16k cost-strict	b64 top4	0.8750	4.8%	2.18×	20.8×

Benchmarks. We evaluate on LongBench-style tasks and RULER synthetic long-context tasks. LongBench examples are JSONL records with fields such as context, input, answers, and length. RULER examples are organized by context length and task, e.g., 4096/niah_single_1.jsonl, with fields input, outputs, length, and index. Current experiments cover LongBench retrieval/summarization subsets and RULER 4k/8k/16k.

Metrics. We report task score, active token or KV ratio, online speedup, end-to-end speedup, and attention-subsystem upper bound:

$$\text{AttentionUpper}(a) \approx \frac{1}{\rho(a; x, q)}. \tag{12}$$

For cache-native experiments, we additionally report query/decode savings, planner overhead, repack overhead, and net component gain.

6 Results

6.1 Block-Size Router End-to-End Results

Table 1 reports prompt-level end-to-end router results. The free small-block router is a high-efficiency candidate but over-selects very small blocks on larger validation. Risk-floor routing constrains the action lattice to the smallest empirically safe action for each task family and length.

6.2 Granularity Frontier

Table 2 shows representative frontier points from the block-size sweep. No single block size dominates: LongBench benefits from 128-token blocks, RULER 4k/8k require 256-token blocks for conservative full accuracy, and RULER 16k has a strong cost-strict 64-token point but requires 512-token blocks for 100% score.

6.3 Cache-Native RiskKV Results

Table 3 summarizes cache-native RiskKV results. These runs use full-context prefill, select active KV pages, apply RoPE-aware repacking, and decode with compact KV. They show that long-context settings convert active KV reduction into online speedup.

Table 3: Cache-native RiskKV results. Online speedup excludes full prompt rebuilding and reflects compact KV decode behavior.

Setting	Method	N	Score	KV ratio	Online	E2E
LongBench m8	Full KV	40	25.00%	100.00%	1.000×	1.000×
LongBench m8	RiskKV	40	32.50%	26.24%	0.988×	0.992×
Mixed13 m2	Full KV	26	69.23%	100.00%	1.000×	1.000×
Mixed13 m2	RiskKV	26	69.23%	23.04%	0.993×	0.996×
RULER 4k m5	RiskKV	40	100.00%	26.30%	0.991×	0.994×
RULER 8k m5	RiskKV	40	100.00%	18.25%	1.075×	1.035×
RULER 16k m3	RiskKV	24	100.00%	8.44%	1.669×	1.184×

Table 4: Cache-native overhead decomposition. Times are per-run aggregate component differences from the full-cache baseline.

Setting	KV	Online	Query saved	Decode saved	Planner+Repack	Net gain
LongBench m8	26.24%	0.988×	11.60 ms	0.96 ms	36.93 ms	-24.37 ms
RULER 4k m5 floor2	26.30%	0.991×	6.22 ms	2.34 ms	27.05 ms	-18.50 ms
RULER 8k m5 floor2	18.25%	1.075×	24.91 ms	155.82 ms	28.56 ms	152.17 ms
RULER 16k m3	8.44%	1.669×	305.68 ms	1425.38 ms	220.04 ms	1511.02 ms

6.4 Attention Subsystem and Overhead

Table 4 decomposes cache-native overhead. At 4k and LongBench lengths, planner and repack overhead can cancel the query/decode savings. At 8k and 16k, savings dominate fixed overhead and the method crosses the break-even point.

7 Ablations

7.1 RoPE-Aware Repacking

Table 5 shows that KV selection alone is insufficient. Naive gather fails even when the selected KV ratio is the same. RoPE-aware repacking restores the compact positions of selected keys and is therefore necessary for cache-native deployment.

7.2 Safety Floor

Multi-evidence tasks require a minimum evidence coverage floor. Table 6 shows that a low-budget conformal policy can fail at 4k m5 by selecting too little evidence, while a k2 floor restores full-level score.

8 Required Experiments to Complete the Submission

The current evidence is strong enough to fix the method story, but a final ICLR submission should fill the following tables. We leave explicit placeholders so that new results can be inserted without restructuring the paper.

9 Discussion and Limitations

RiskKV-Block is strongest when the task provides a query that identifies sparse evidence in a long context. It is therefore naturally suited to long-context QA, retrieval, multi-hop evidence lookup, and needle-style stress tests. It is less directly suited to pure streaming language modeling, where no explicit query is available. For summarization, counting, scientific QA, and code completion, sparse block retrieval may miss global information; these cases require summary, aggregation, or full-memory fallback.

Table 5: RoPE-aware repacking ablation. Naive gathering fails at the same KV ratio; RoPE repacking recovers full-cache behavior.

Setting	Method	Score	KV ratio	Online	E2E
Mixed13 m1	Full KV	69.23%	100.00%	1.000×	1.000×
Mixed13 m1	Naive gather + compact query	38.46%	26.08%	1.051×	1.033×
Mixed13 m1	RoPE repack + compact query	69.23%	26.08%	1.046×	1.030×
RULER 8k m3	Naive gather + compact query	37.50%	13.14%	1.112×	1.052×
RULER 8k m3	RoPE repack + compact query	100.00%	13.14%	1.108×	1.051×
RULER 16k m2	Naive gather + compact query	12.50%	6.41%	1.719×	1.194×
RULER 16k m2	RoPE repack + compact query	100.00%	6.41%	1.712×	1.193×

Table 6: Safety-floor ablation on RULER m5.

Setting	Score	KV ratio	Online
4k conformal no floor	95.00%	14.61%	0.987×
4k conformal floor2	100.00%	26.30%	0.991×
8k conformal floor2	100.00%	18.25%	1.075×

The current block scorer is intentionally simple and interpretable. Replacing it with a learned retriever or using model-internal attention probes may improve recall. The current router is trained from sweep labels; stronger calibration and conformal risk guarantees remain important future work. Finally, the prompt-level router results and cache-native KV results should be unified into one optimized serving implementation for the final submission.

10 Conclusion

We introduced RiskKV-Block, a risk-constrained memory granularity router for long-context inference. The method jointly selects block size, top-k evidence budget, and fallback policy, and can be deployed as either prompt-level selected spans or cache-native KV selection with RoPE-aware repacking. Current results show that adaptive block-size routing improves score while reducing active tokens to 11.04% and achieving 1.756× end-to-end speedup, and cache-native experiments demonstrate that RoPE-aware repacking is essential for compact KV decoding. The results support a broader conclusion: efficient long-context inference should adapt not only how much memory to keep, but also at what granularity to keep it.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508, 2023.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, Wen Xiao, Qi Yuan, and Wenxuan Zhang. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. arXiv preprint arXiv:2406.02069, 2024.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? arXiv preprint arXiv:2404.06654, 2024.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharath Venkitesh, Andrea Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469, 2024.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453, 2023.

Table 7: [TODO: Main multi-model benchmark table. Fill after running router v2 on full or large-sample LongBench/RULER splits.]

Model	Benchmark	Method	Score	Token/KV	E2E	Online	Memory
Qwen3-8B	LongBench	Full	–	100%	1.00×	1.00×	–
Qwen3-8B	LongBench	RiskKV-Block	–	–	–	–	–
Qwen3-8B	RULER full	RiskKV-Block	–	–	–	–	–
Llama-3.1-8B	LongBench	RiskKV-Block	–	–	–	–	–
Mistral-7B	LongBench	RiskKV-Block	–	–	–	–	–

Table 8: [TODO: Baseline comparison table. Include official or reproduced SnapKV, H2O, PyramidKV, StreamingLLM/sliding-window, and fixed-block baselines.]

Method	LongBench	RULER 8k	RULER 16k	Token/KV	E2E	Notes
Full KV/raw	–	–	–	100%	1.00×	–
Sliding window	–	–	–	–	–	–
H_2O	–	–	–	–	–	–
SnapKV	–	–	–	–	–	–
PyramidKV	–	–	–	–	–	–
RiskKV-Block	–	–	–	–	–	–

Dongjie Yang, Xiao Han, Yan Gao, Yao Hu, Shiyang Zhang, and Hai Zhao. Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference. arXiv preprint arXiv:2405.12532, 2024.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048, 2023.

Zhenyu Zhou, Xuefei Ning, Ke Hong, Tianchen Fu, Jiaming Xu, Shiyao Li, Yiping Lou, Liyue Wang, Zhihang Yuan, Xiuhong Li, Guohao Yan, Guohao Dai, Huazhong Wang, Yu Yang, and Yu Wang. Dynamickv: Task-aware adaptive kv cache compression for long context llms. arXiv preprint arXiv:2412.14838, 2024.

A Implementation Details

Content words. The current lexical scorer extracts English words using the regular expression [A-Za-z][A-Za-z'-]{2,}, lowercases them, and removes stopwords. The content overlap is the size of the set intersection between query words and block words.

Identifier matching. The identifier extractor uses [A-Za-z]*\d+[A-Za-z0-9-]*, capturing digit-bearing keys such as 8090293, case123, and A12B. The identifier overlap is weighted by $\lambda = 3$ in Eq. 7.

Block order. Top-scoring blocks are selected by score, but the selected blocks are emitted in original context order. This preserves local discourse order and avoids confusing the downstream model.

B Additional Placeholder Tables

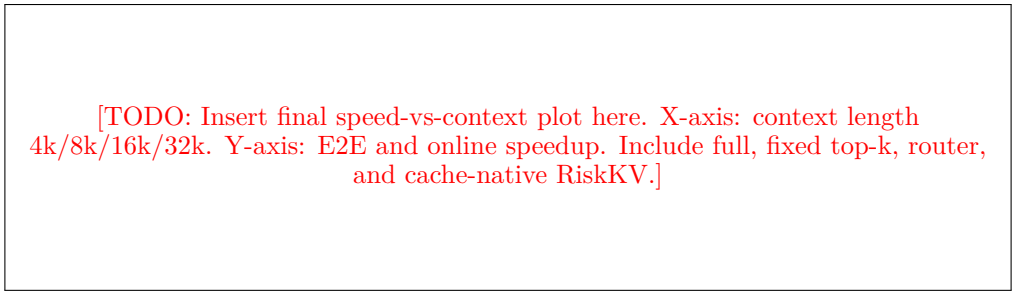

Figure 2: Placeholder for scaling figure.

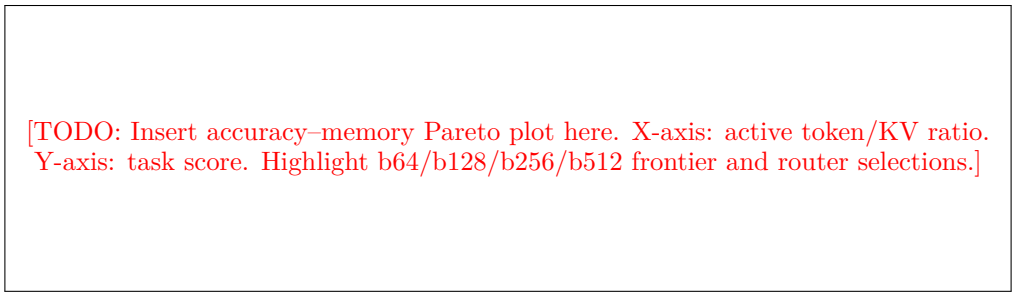

Figure 3: Placeholder for Pareto frontier figure.

Table 9: Router variants. Exact label accuracy is conservative because nearby actions often preserve the same task score with slightly different token ratios.

Router	Score	Token	E2E	Label acc.
Router v1	0.8535	11.04%	1.756×	72.2%
Free small-block v2	0.8634	7.82%	1.80×	50.0%
Risk-floor v2 m10	0.8398	18.11%	1.58×	n/a

Table 10: [TODO: Ablation table for router components.]

Ablation	Score	Token	E2E
Full RiskKV-Block	–	–	–
w/o block-size routing	–	–	–
w/o fallback	–	–	–
w/o identifier overlap	–	–	–
w/o retrieval stability features	–	–	–